

A REFERENCE HANDBOOK

Python Object-Oriented Programming

100 Practical Coding Problems & Solutions in Python



CONTENTS

01	Classes & Objects Basics	10 problems
02	Encapsulation	10 problems
03	Inheritance	15 problems
04	Polymorphism	12 problems
05	Abstraction	10 problems
06	Magic Methods & Operator Overloading	12 problems
07	Static & Class Members	8 problems
08	Design Patterns	10 problems
09	Composition & Mixins	8 problems
10	Exception Handling in OOP	5 problems

Classes & Objects Basics

001 Create a basic class with attributes

Create a class 'Book' with attributes title, author, and price. Create an instance and print its details.

SOLUTION – PYTHON

```
class Book:
    def __init__(self, title, author, price):
        self.title = title
        self.author = author
        self.price = price

    def display(self):
        print(f"{self.title} by {self.author} - 
        ${self.price:.2f}")

b = Book("Clean Code", "Robert C. Martin", 34.99)
b.display()
# Output: Clean Code by Robert C. Martin - $34.99
```

002 Instance counter

Create a class 'Employee' that keeps track of how many Employee objects have been created using a class attribute.

SOLUTION – PYTHON

```
class Employee:
    count = 0

    def __init__(self, name):
        self.name = name
        Employee.count += 1

e1 = Employee("Alice")
e2 = Employee("Bob")
e3 = Employee("Carol")
print(f"Total employees: {Employee.count}")
# Output: Total employees: 3
```

003 Bank account class with deposit/withdraw

Create a 'BankAccount' class with a balance, and methods deposit() and withdraw() that validate the amount and prevent overdrafts.

SOLUTION – PYTHON

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit amount must be
positive")
        self.balance += amount

    def withdraw(self, amount):
        if amount <= 0:
            raise ValueError("Withdrawal amount must be
positive")
        if amount > self.balance:
            raise ValueError("Insufficient funds")
        self.balance -= amount

acc = BankAccount("Priya", 100)
acc.deposit(50)
acc.withdraw(30)
print(acc.balance) # 120
```

004 Rectangle class with area and perimeter

Create a 'Rectangle' class with width and height, and methods to compute area and perimeter.

SOLUTION – PYTHON

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

r = Rectangle(4, 5)
print(r.area())      # 20
print(r.perimeter()) # 18
```

005 Compare two objects for equality by attribute

Create a 'Point' class with x and y attributes, and implement equality comparison so two points with the same coordinates are considered equal.

SOLUTION – PYTHON

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __eq__(self, other):
        if not isinstance(other, Point):
            return NotImplemented
        return self.x == other.x and self.y == other.y

p1 = Point(1, 2)
p2 = Point(1, 2)
p3 = Point(3, 4)
print(p1 == p2) # True
print(p1 == p3) # False
```

006 Student grade average calculator

Create a 'Student' class that stores a list of grades and has a method to compute the average grade.

SOLUTION – PYTHON

```
class Student:
    def __init__(self, name):
        self.name = name
        self.grades = []

    def add_grade(self, grade):
        self.grades.append(grade)

    def average(self):
        if not self.grades:
            return 0
        return sum(self.grades) / len(self.grades)

s = Student("Kenji")
s.add_grade(85)
s.add_grade(92)
s.add_grade(78)
print(f"{s.average():.2f}") # 85.00
```

007 Object with a dynamic `__dict__` attribute

Write a function that takes an object and prints all its instance attribute names and values using `__dict__`.

SOLUTION – PYTHON

```
class Car:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year

def print_attributes(obj):
    for key, value in obj.__dict__.items():
        print(f"{key}: {value}")

c = Car("Toyota", "Corolla", 2022)
print_attributes(c)
# make: Toyota
# model: Corolla
# year: 2022
```

008 Shallow copy vs deep copy demonstration

Demonstrate the difference between a shallow copy and a deep copy using a class that contains a mutable list attribute.

SOLUTION – PYTHON

```
import copy

class Team:
    def __init__(self, members):
        self.members = members

original = Team(["Alice", "Bob"])
shallow = copy.copy(original)
deep = copy.deepcopy(original)

shallow.members.append("Carol")
print(original.members)  # ['Alice', 'Bob', 'Carol'] (shared list)
print(deep.members)      # ['Alice', 'Bob'] (independent list)
```

009 Using `__slots__` to restrict attributes

Create a 'Coordinate' class that uses `__slots__` to only allow x and y attributes, and show that adding another attribute raises an error.

SOLUTION – PYTHON

```
class Coordinate:
    __slots__ = ("x", "y")

    def __init__(self, x, y):
        self.x = x
        self.y = y

c = Coordinate(1, 2)
try:
    c.z = 3
except AttributeError as e:
    print(f"Error: {e}")
# Error: 'Coordinate' object has no attribute 'z'
```

010 Class representing a 2D vector with basic arithmetic

Create a 'Vector2D' class with x and y, and a method to compute its magnitude.

SOLUTION – PYTHON

```
import math

class Vector2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def magnitude(self):
        return math.sqrt(self.x ** 2 + self.y ** 2)

v = Vector2D(3, 4)
print(v.magnitude()) # 5.0
```

Encapsulation

011 Private attribute with getter and setter

Create a 'Person' class with a private '__age' attribute, exposing it through a getter and a setter that validates the age is non-negative.

SOLUTION – PYTHON

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.__age = age

    def get_age(self):
        return self.__age

    def set_age(self, age):
        if age < 0:
            raise ValueError("Age cannot be negative")
        self.__age = age

p = Person("Nina", 30)
print(p.get_age()) # 30
p.set_age(31)
print(p.get_age()) # 31
```

012 Using @property for validated attribute access

Rewrite a 'Temperature' class so that its 'celsius' attribute can be read and written using dot notation, but raises an error for temperatures below absolute zero.

SOLUTION – PYTHON

```
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius # uses the setter

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("Temperature below absolute zero
is not possible")
        self._celsius = value

t = Temperature(25)
print(t.celsius) # 25
try:
    t.celsius = -300
except ValueError as e:
    print(f"Error: {e}")
# Error: Temperature below absolute zero is not possible
```

013 Read-only computed property

Create a 'Circle' class with a radius, exposing a read-only 'area' property computed from the radius.

SOLUTION – PYTHON

```
import math

class Circle:
    def __init__(self, radius):
        self.radius = radius

    @property
    def area(self):
        return math.pi * self.radius ** 2

c = Circle(3)
print(round(c.area, 2)) # 28.27
try:
    c.area = 100
except AttributeError as e:
    print(f"Error: {e}")
# Error: can't set attribute 'area'
```

014 Name-mangled private attribute demo

Demonstrate Python's name mangling by creating a class with a '__balance' attribute and accessing it from outside using the mangled name.

SOLUTION – PYTHON

```
class Wallet:
    def __init__(self, balance):
        self.__balance = balance

w = Wallet(100)
# print(w.__balance)           # would raise AttributeError
print(w._Wallet__balance)     # 100 (name-mangled access,
                               # not recommended)
```

015 Encapsulated inventory item with quantity validation

Create an 'InventoryItem' class where the quantity cannot be set directly to a negative number, using a property setter.

SOLUTION – PYTHON

```
class InventoryItem:
    def __init__(self, name, quantity):
        self.name = name
        self.quantity = quantity

    @property
    def quantity(self):
        return self._quantity

    @quantity.setter
    def quantity(self, value):
        if value < 0:
            raise ValueError("Quantity cannot be negative")
        self._quantity = value

item = InventoryItem("Widget", 10)
item.quantity -= 5
print(item.quantity)  # 5
try:
    item.quantity = -1
except ValueError as e:
    print(f"Error: {e}")
```

016 Encapsulated password field with masking

Create a 'UserAccount' class that stores a password privately and exposes only a masked version through a property.

SOLUTION – PYTHON

```
class UserAccount:
    def __init__(self, username, password):
        self.username = username
        self.__password = password

    @property
    def masked_password(self):
        return "*" * len(self.__password)

    def check_password(self, attempt):
        return attempt == self.__password

u = UserAccount("dev_user", "s3cr3t!")
print(u.masked_password)          # *****
print(u.check_password("s3cr3t!")) # True
```

017 Immutable point class using properties

Create a 'Point' class whose x and y coordinates cannot be changed after creation (read-only properties).

SOLUTION – PYTHON

```
class Point:
    def __init__(self, x, y):
        self._x = x
        self._y = y

    @property
    def x(self):
        return self._x

    @property
    def y(self):
        return self._y

    def __repr__(self):
        return f"Point({self._x}, {self._y})"

p = Point(2, 3)
print(p)  # Point(2, 3)
try:
    p.x = 10
except AttributeError as e:
    print(f"Error: {e}")
```

018 Encapsulated Celsius/Fahrenheit conversion

Create a 'Weather' class storing temperature in Celsius internally, but exposing a 'fahrenheit' property to get and set the temperature in Fahrenheit.

SOLUTION – PYTHON

```
class Weather:
    def __init__(self, celsius=0):
        self.celsius = celsius

    @property
    def fahrenheit(self):
        return self.celsius * 9 / 5 + 32

    @fahrenheit.setter
    def fahrenheit(self, value):
        self.celsius = (value - 32) * 5 / 9

w = Weather(celsius=20)
print(w.fahrenheit)  # 68.0
w.fahrenheit = 98.6
print(round(w.celsius, 1))  # 37.0
```

019 Validated email field using a property

Create a 'Contact' class whose 'email' property validates that the value contains an '@' symbol before allowing assignment.

SOLUTION – PYTHON

```
class Contact:
    def __init__(self, name, email):
        self.name = name
        self.email = email

    @property
    def email(self):
        return self._email

    @email.setter
    def email(self, value):
        if "@" not in value:
            raise ValueError("Invalid email address")
        self._email = value

c = Contact("Sam", "sam@example.com")
print(c.email)
try:
    c.email = "not-an-email"
except ValueError as e:
    print(f"Error: {e}")
```

020 Encapsulated stack with controlled access

Implement a 'Stack' class that hides its underlying list and exposes only push, pop, peek, and is_empty methods.

SOLUTION – PYTHON

```
class Stack:
    def __init__(self):
        self.__items = []

    def push(self, item):
        self.__items.append(item)

    def pop(self):
        if self.is_empty():
            raise IndexError("pop from empty stack")
        return self.__items.pop()

    def peek(self):
        if self.is_empty():
            raise IndexError("peek from empty stack")
        return self.__items[-1]

    def is_empty(self):
        return len(self.__items) == 0

s = Stack()
s.push(1)
s.push(2)
print(s.peek()) # 2
print(s.pop()) # 2
print(s.is_empty()) # False
```

Inheritance

021 Basic single inheritance

Create an 'Animal' base class with a `speak()` method, and a 'Dog' subclass that overrides `speak()`.

SOLUTION – PYTHON

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return f"{self.name} makes a sound."

class Dog(Animal):
    def speak(self):
        return f"{self.name} says Woof!"

a = Animal("Generic Animal")
d = Dog("Rex")
print(a.speak()) # Generic Animal makes a sound.
print(d.speak()) # Rex says Woof!
```

022 Using super() to extend parent constructor

Create a 'Vehicle' base class and a 'Car' subclass that adds a 'doors' attribute while still initializing the parent's attributes via super().

SOLUTION – PYTHON

```
class Vehicle:
    def __init__(self, make, model):
        self.make = make
        self.model = model

class Car(Vehicle):
    def __init__(self, make, model, doors):
        super().__init__(make, model)
        self.doors = doors

    def __repr__(self):
        return f"{self.make} {self.model} ({self.doors} doors)"

c = Car("Honda", "Civic", 4)
print(c)  # Honda Civic (4 doors)
```

023 Multilevel inheritance chain

Create a three-level hierarchy: 'LivingBeing' -> 'Animal' -> 'Dog', each adding new behavior.

SOLUTION – PYTHON

```
class LivingBeing:
    def breathe(self):
        return "Breathing..."

class Animal(LivingBeing):
    def move(self):
        return "Moving..."

class Dog(Animal):
    def bark(self):
        return "Barking!"

d = Dog()
print(d.breathe()) # Breathing...
print(d.move())    # Moving...
print(d.bark())    # Barking!
```

024 Hierarchical inheritance with a shared parent

Create a 'Shape' base class with an abstract-like area() method, and 'Circle' and 'Square' subclasses that each implement it differently.

SOLUTION – PYTHON

```
class Shape:
    def area(self):
        raise NotImplementedError("Subclasses must implement area()")

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius ** 2

class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side ** 2

shapes = [Circle(3), Square(4)]
for s in shapes:
    print(round(s.area(), 2))
# 28.27
# 16
```

025 Multiple inheritance and MRO inspection

Create classes A, B, and C where C inherits from both A and B, and print C's Method Resolution Order.

SOLUTION – PYTHON

```
class A:
    def greet(self):
        return "Hello from A"

class B:
    def greet(self):
        return "Hello from B"

class C(A, B):
    pass

c = C()
print(c.greet())           # Hello from A (A comes first in
                           # MRO)
print([cls.__name__ for cls in C.__mro__])
# ['C', 'A', 'B', 'object']
```

026 Diamond inheritance resolved with super()

Demonstrate the diamond inheritance pattern where D inherits from B and C, which both inherit from A, and show that A's method runs only once when using `super()` cooperatively.

SOLUTION – PYTHON

```
class A:
    def __init__(self):
        print("A init")
        self.value = 1

class B(A):
    def __init__(self):
        super().__init__()
        print("B init")

class C(A):
    def __init__(self):
        super().__init__()
        print("C init")

class D(B, C):
    def __init__(self):
        super().__init__()
        print("D init")

D()
# A init
# C init
# B init
# D init
```

027 Overriding a method and still calling the parent version

Create a 'Logger' base class with a log() method, and a 'TimestampLogger' subclass that overrides log() to add a timestamp while still calling the original log() method.

SOLUTION – PYTHON

```
import datetime

class Logger:
    def log(self, message):
        print(f"LOG: {message}")

class TimestampLogger(Logger):
    def log(self, message):
        timestamp = datetime.datetime(2026, 1, 1, 12, 0, 0).isoformat()
        super().log(f"[{timestamp}] {message}")

logger = TimestampLogger()
logger.log("Application started")
# LOG: [2026-01-01T12:00:00] Application started
```

028 Employee/Manager hierarchy with salary calculation

Create an 'Employee' class with a base salary calculation, and a 'Manager' subclass that adds a bonus on top of the inherited calculation.

SOLUTION – PYTHON

```
class Employee:
    def __init__(self, name, base_salary):
        self.name = name
        self.base_salary = base_salary

    def calculate_salary(self):
        return self.base_salary

class Manager(Employee):
    def __init__(self, name, base_salary, bonus):
        super().__init__(name, base_salary)
        self.bonus = bonus

    def calculate_salary(self):
        return super().calculate_salary() + self.bonus

e = Employee("Tom", 50000)
m = Manager("Ana", 70000, 15000)
print(e.calculate_salary()) # 50000
print(m.calculate_salary()) # 85000
```

029 Abstract base with concrete shared behavior and required overrides

Create a 'PaymentMethod' base class with a shared validate() method and an abstract-like process_payment() method that subclasses must implement.

SOLUTION – PYTHON

```
class PaymentMethod:
    def validate(self, amount):
        if amount <= 0:
            raise ValueError("Amount must be positive")
        return True

    def process_payment(self, amount):
        raise NotImplementedError("Subclasses must implement
process_payment()")

class CreditCardPayment(PaymentMethod):
    def process_payment(self, amount):
        self.validate(amount)
        return f"Charged ${amount:.2f} to credit card"

class PayPalPayment(PaymentMethod):
    def process_payment(self, amount):
        self.validate(amount)
        return f"Charged ${amount:.2f} via PayPal"

for method in (CreditCardPayment(), PayPalPayment()):
    print(method.process_payment(49.99))
```

030 Mixin combined with inheritance

Create a 'JSONSerializableMixin' that adds a to_json() method, and combine it with a 'Product' class using multiple inheritance.

SOLUTION – PYTHON

```
import json

class JSONSerializableMixin:
    def to_json(self):
        return json.dumps(self.__dict__)

class Product(JSONSerializableMixin):
    def __init__(self, name, price):
        self.name = name
        self.price = price

p = Product("Keyboard", 79.99)
print(p.to_json())
# {"name": "Keyboard", "price": 79.99}
```

031 Preventing further subclassing

Create a 'FinalConfig' class that raises an error if anyone tries to subclass it, using `__init_subclass__`.

SOLUTION – PYTHON

```
class FinalConfig:
    def __init_subclass__(cls, **kwargs):
        raise TypeError(f"Cannot subclass FinalConfig
(attempted by {cls.__name__})")

try:
    class ExtendedConfig(FinalConfig):
        pass
except TypeError as e:
    print(f"Error: {e}")
# Error: Cannot subclass FinalConfig (attempted by
ExtendedConfig)
```

032 Shape hierarchy with a common constructor pattern

Create an abstract-style 'Shape' base storing a color attribute shared by all subclasses like 'Triangle' and 'Hexagon'.

SOLUTION – PYTHON

```
class Shape:
    def __init__(self, color):
        self.color = color

    def describe(self):
        return f"A {self.color} shape"

class Triangle(Shape):
    def __init__(self, color, base, height):
        super().__init__(color)
        self.base = base
        self.height = height

    def area(self):
        return 0.5 * self.base * self.height

t = Triangle("red", 6, 4)
print(t.describe()) # A red shape
print(t.area())     # 12.0
```

033 Vehicle rental hierarchy with polymorphic pricing

Create a 'RentalVehicle' base class and 'Car' and 'Motorcycle' subclasses, each with its own daily rate calculation formula.

SOLUTION – PYTHON

```
class RentalVehicle:
    def __init__(self, daily_rate):
        self.daily_rate = daily_rate

    def total_cost(self, days):
        return self.daily_rate * days

class Car(RentalVehicle):
    def total_cost(self, days):
        base = super().total_cost(days)
        return base + 10 * days # insurance surcharge

class Motorcycle(RentalVehicle):
    def total_cost(self, days):
        return super().total_cost(days) * 0.9 # discount

c = Car(50)
m = Motorcycle(30)
print(c.total_cost(3)) # 180 (150 base + 30 insurance surcharge)
print(m.total_cost(3)) # 81.0
```

034 Extending a built-in type

Create a 'UniqueList' class that inherits from Python's built-in list but overrides `append()` to avoid adding duplicate values.

SOLUTION – PYTHON

```
class UniqueList(list):
    def append(self, item):
        if item not in self:
            super().append(item)

ul = UniqueList()
ul.append(1)
ul.append(2)
ul.append(1)
print(ul)  # [1, 2]
```

035 Interface-like abstract base enforced across a hierarchy

Create a 'Comparable' style base class that requires subclasses to implement `compare_to()`, and use it to sort custom objects.

SOLUTION – PYTHON

```
class Comparable:
    def compare_to(self, other):
        raise NotImplementedError

class Money(Comparable):
    def __init__(self, amount):
        self.amount = amount

    def compare_to(self, other):
        return (self.amount > other.amount) - (self.amount <
other.amount)

    def __repr__(self):
        return f"${self.amount}"

import functools
amounts = [Money(50), Money(10), Money(30)]
amounts.sort(key=functools.cmp_to_key(lambda a, b:
a.compare_to(b)))
print(amounts) # [$10, $30, $50]
```

CHAPTER 04

Polymorphism

036 Polymorphic speak() across multiple subclasses

Create Animal subclasses Cat, Dog, and Bird that each override speak(), then call speak() polymorphically in a loop.

SOLUTION – PYTHON

```
class Animal:
    def speak(self):
        raise NotImplementedError

class Cat(Animal):
    def speak(self):
        return "Meow"

class Dog(Animal):
    def speak(self):
        return "Woof"

class Bird(Animal):
    def speak(self):
        return "Tweet"

for animal in (Cat(), Dog(), Bird()):
    print(animal.speak())
# Meow
# Woof
# Tweet
```

037 Duck typing with unrelated classes

Create two unrelated classes, 'Duck' and 'Robot', that both implement a quack() method, and call quack() on each without checking their type.

SOLUTION – PYTHON

```
class Duck:
    def quack(self):
        return "Quack!"

class Robot:
    def quack(self):
        return "BEEP - simulated quack"

def make_it_quack(entity):
    print(entity.quack())

make_it_quack(Duck())
make_it_quack(Robot())
# Quack!
# BEEP - simulated quack
```

038 Polymorphic total_area function over shapes

Write a function `total_area(shapes)` that sums the `area()` of any list of shape objects, regardless of their concrete class.

SOLUTION – PYTHON

```
class Circle:
    def __init__(self, r):
        self.r = r

    def area(self):
        return 3.14159 * self.r ** 2

class Rectangle:
    def __init__(self, w, h):
        self.w, self.h = w, h

    def area(self):
        return self.w * self.h

def total_area(shapes):
    return sum(shape.area() for shape in shapes)

shapes = [Circle(2), Rectangle(3, 4)]
print(round(total_area(shapes), 2)) # 24.57
```

039 Operator overloading enabling polymorphic + across custom types

Create a 'Money' class that overloads + so two Money objects, or a Money and a plain number, can be added together.

SOLUTION – PYTHON

```
class Money:
    def __init__(self, amount):
        self.amount = amount

    def __add__(self, other):
        if isinstance(other, Money):
            return Money(self.amount + other.amount)
        elif isinstance(other, (int, float)):
            return Money(self.amount + other)
        return NotImplemented

    def __repr__(self):
        return f"${self.amount:.2f}"

m1 = Money(10)
m2 = Money(5.5)
print(m1 + m2)    # $15.50
print(m1 + 4)     # $14.00
```

040 Using functools.singledispatch for type-based dispatch

Write a generic 'describe' function that behaves differently depending on the runtime type of its argument, using functools.singledispatch.

SOLUTION – PYTHON

```
from functools import singledispatch

@singledispatch
def describe(value):
    return f"A value: {value}"

@describe.register
def _(value: int):
    return f"An integer: {value}"

@describe.register
def _(value: str):
    return f"A string of length {len(value)}: &#x27;{value}&#x27;"

print(describe(42))          # An integer: 42
print(describe("hello"))     # A string of length 5: 'hello'
print(describe(3.14))        # A value: 3.14
```

041 Polymorphism with an abstract Notification base

Create a 'Notification' base class and 'EmailNotification'/'SMSNotification' subclasses, each implementing send() differently, then send all notifications from a single list.

SOLUTION – PYTHON

```
class Notification:
    def send(self, message):
        raise NotImplementedError

class EmailNotification(Notification):
    def send(self, message):
        return f"Emailing: {message}"

class SMSNotification(Notification):
    def send(self, message):
        return f"Texting: {message}"

notifications = [EmailNotification(), SMSNotification()]
for n in notifications:
    print(n.send("Your order has shipped"))
```

042 Overloading `__len__` for polymorphic `len()` support

Create a 'Playlist' class that stores songs and supports Python's built-in `len()` function by implementing `__len__`.

SOLUTION – PYTHON

```
class Playlist:
    def __init__(self, songs=None):
        self.songs = songs or []

    def __len__(self):
        return len(self.songs)

p = Playlist(["Song A", "Song B", "Song C"])
print(len(p)) # 3
```

043 Comparable objects sorted polymorphically

Create a 'Product' class with `__lt__` implemented so a list of Products can be sorted directly using the built-in `sorted()` function.

SOLUTION – PYTHON

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def __lt__(self, other):
        return self.price < other.price

    def __repr__(self):
        return f"{self.name}(${self.price})"

products = [Product("Mug", 12), Product("Pen", 2),
            Product("Bag", 25)]
print(sorted(products))
# [Pen($2), Mug($12), Bag($25)]
```

044 Polymorphic file-like objects sharing a write interface

Create two classes, 'ConsoleWriter' and 'ListWriter', that both implement a write(text) method, and use them interchangeably in a shared logging function.

SOLUTION – PYTHON

```
class ConsoleWriter:
    def write(self, text):
        print(f"CONSOLE: {text}")

class ListWriter:
    def __init__(self):
        self.lines = []

    def write(self, text):
        self.lines.append(text)

def log_message(writer, message):
    writer.write(message)

cw = ConsoleWriter()
lw = ListWriter()
log_message(cw, "Startup complete")
log_message(lw, "Startup complete")
print(lw.lines)  # ['Startup complete']
```

045 Overriding `__call__` for polymorphic callable objects

Create two classes, 'Doubler' and 'Squarer', that are both callable (implement `__call__`), and apply them polymorphically inside a `map()`-like loop.

SOLUTION – PYTHON

```
class Doubler:
    def __call__(self, x):
        return x * 2

class Squarer:
    def __call__(self, x):
        return x ** 2

def apply_to_all(func, values):
    return [func(v) for v in values]

print(apply_to_all(Doubler(), [1, 2, 3])) # [2, 4, 6]
print(apply_to_all(Squarer(), [1, 2, 3])) # [1, 4, 9]
```

046 Polymorphic discount strategies applied to an order

Create a 'DiscountStrategy' base class with a couple of subclasses implementing apply(price) differently, then apply whichever strategy is passed to an Order.

SOLUTION – PYTHON

```

class DiscountStrategy:
    def apply(self, price):
        raise NotImplementedError

class PercentageDiscount(DiscountStrategy):
    def __init__(self, percent):
        self.percent = percent

    def apply(self, price):
        return price * (1 - self.percent / 100)

class FlatDiscount(DiscountStrategy):
    def __init__(self, amount):
        self.amount = amount

    def apply(self, price):
        return max(0, price - self.amount)

class Order:
    def __init__(self, price, discount: DiscountStrategy):
        self.price = price
        self.discount = discount

    def final_price(self):
        return self.discount.apply(self.price)

o1 = Order(100, PercentageDiscount(20))
o2 = Order(100, FlatDiscount(15))
print(o1.final_price()) # 80.0
print(o2.final_price()) # 85

```

047 Polymorphic iterator objects

Create two custom iterable classes, 'EvenNumbers' and 'OddNumbers', that both implement `__iter__` and `__next__`, and consume them interchangeably in a for-loop.

SOLUTION – PYTHON

```
class EvenNumbers:
    def __init__(self, limit):
        self.limit = limit
        self.n = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.n >= self.limit:
            raise StopIteration
        value = self.n
        self.n += 2
        return value

class OddNumbers:
    def __init__(self, limit):
        self.limit = limit
        self.n = 1

    def __iter__(self):
        return self

    def __next__(self):
        if self.n >= self.limit:
            raise StopIteration
        value = self.n
        self.n += 2
        return value

for seq in (EvenNumbers(6), OddNumbers(6)):
    print(list(seq))
# [0, 2, 4]
# [1, 3, 5]
```

Abstraction

048 Abstract base class with abstractmethod

Create an abstract 'Shape' class using ABC and abstractmethod that forces subclasses to implement area(), and show that instantiating Shape directly fails.

SOLUTION – PYTHON

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side ** 2

try:
    s = Shape()
except TypeError as e:
    print(f"Error: {e}")

sq = Square(4)
print(sq.area()) # 16
```

049 Abstract class with both abstract and concrete methods

Create an abstract 'Employee' class with an abstract `calculate_pay()` method and a concrete `display_info()` method shared by all subclasses.

SOLUTION – PYTHON

```
from abc import ABC, abstractmethod

class Employee(ABC):
    def __init__(self, name):
        self.name = name

    @abstractmethod
    def calculate_pay(self):
        pass

    def display_info(self):
        print(f"{self.name}: ${self.calculate_pay():.2f}")

class HourlyEmployee(Employee):
    def __init__(self, name, hours, rate):
        super().__init__(name)
        self.hours = hours
        self.rate = rate

    def calculate_pay(self):
        return self.hours * self.rate

he = HourlyEmployee("Lena", 40, 25)
he.display_info() # Lena: $1000.00
```

050 Abstract database connector interface

Create an abstract 'DatabaseConnector' with abstract connect() and disconnect() methods, implemented differently by 'MySQLConnector' and 'PostgresConnector'.

SOLUTION – PYTHON

```
from abc import ABC, abstractmethod

class DatabaseConnector(ABC):
    @abstractmethod
    def connect(self):
        pass

    @abstractmethod
    def disconnect(self):
        pass

class MySQLConnector(DatabaseConnector):
    def connect(self):
        return "Connected to MySQL"

    def disconnect(self):
        return "Disconnected from MySQL"

class PostgresConnector(DatabaseConnector):
    def connect(self):
        return "Connected to PostgreSQL"

    def disconnect(self):
        return "Disconnected from PostgreSQL"

for connector in (MySQLConnector(), PostgresConnector()):
    print(connector.connect())
```

051 Enforcing an abstract property

Create an abstract 'Account' class that requires subclasses to implement an 'interest_rate' property.

SOLUTION – PYTHON

```
from abc import ABC, abstractmethod

class Account(ABC):
    @property
    @abstractmethod
    def interest_rate(self):
        pass

class SavingsAccount(Account):
    @property
    def interest_rate(self):
        return 0.03

class CheckingAccount(Account):
    @property
    def interest_rate(self):
        return 0.0

print(SavingsAccount().interest_rate) # 0.03
print(CheckingAccount().interest_rate) # 0.0
```

052 Incomplete subclass raises TypeError

Show what happens when a subclass of an abstract class fails to implement all required abstract methods.

SOLUTION – PYTHON

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

class Incomplete(Shape):
    def area(self):
        return 0
    # perimeter() is missing

try:
    Incomplete()
except TypeError as e:
    print(f"Error: {e}")
# Error: Can't instantiate abstract class Incomplete with
abstract method perimeter
```

053 Abstract report generator template

Create an abstract 'ReportGenerator' whose generate() method defines the overall steps, calling abstract fetch_data() and format_data() methods that subclasses must implement (Template Method pattern).

SOLUTION – PYTHON

```
from abc import ABC, abstractmethod

class ReportGenerator(ABC):
    def generate(self):
        data = self.fetch_data()
        formatted = self.format_data(data)
        return f"REPORT:\n{formatted}"

    @abstractmethod
    def fetch_data(self):
        pass

    @abstractmethod
    def format_data(self, data):
        pass

class SalesReport(ReportGenerator):
    def fetch_data(self):
        return [100, 200, 300]

    def format_data(self, data):
        return f"Total sales: {sum(data)}"

print(SalesReport().generate())
# REPORT:
# Total sales: 600
```

054 Using typing.Protocol for structural abstraction

Define a 'Flyable' Protocol requiring a fly() method, and write a function that accepts any object matching that structure without formal inheritance.

SOLUTION – PYTHON

```
from typing import Protocol

class Flyable(Protocol):
    def fly(self) -> str:
        ...

class Airplane:
    def fly(self):
        return "Airplane is flying"

class Bird:
    def fly(self):
        return "Bird is flying"

def make_it_fly(entity: Flyable):
    print(entity.fly())

make_it_fly(Airplane())
make_it_fly(Bird())
```

055 Abstract shape hierarchy with a shared scale() method

Create an abstract 'Shape2D' class with an abstract area() method and a concrete scale() method that multiplies a stored scale factor and reports it.

SOLUTION – PYTHON

```
from abc import ABC, abstractmethod

class Shape2D(ABC):
    def __init__(self):
        self.scale_factor = 1.0

    @abstractmethod
    def area(self):
        pass

    def scale(self, factor):
        self.scale_factor *= factor
        return f"Scaled by {factor}, cumulative factor: {self.scale_factor}"

class Square(Shape2D):
    def __init__(self, side):
        super().__init__()
        self.side = side

    def area(self):
        return (self.side * self.scale_factor) ** 2

sq = Square(5)
print(sq.scale(2))    # Scaled by 2, cumulative factor: 2.0
print(sq.area())     # 100.0
```

056 Abstract state machine interface

Create an abstract 'State' class with an abstract handle() method, used to implement a simple traffic-light state machine.

SOLUTION – PYTHON

```
from abc import ABC, abstractmethod

class State(ABC):
    @abstractmethod
    def handle(self):
        pass

class RedLight(State):
    def handle(self):
        return "Stop"

class GreenLight(State):
    def handle(self):
        return "Go"

class TrafficLight:
    def __init__(self, state: State):
        self.state = state

    def change_state(self, state: State):
        self.state = state

    def report(self):
        return self.state.handle()

light = TrafficLight(RedLight())
print(light.report()) # Stop
light.change_state(GreenLight())
print(light.report()) # Go
```

057 Abstract Shape hierarchy with __str__ shared implementation

Create an abstract 'Shape' class that provides a shared __str__ implementation using an abstract name() method and abstract area() method.

SOLUTION – PYTHON

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def name(self):
        pass

    @abstractmethod
    def area(self):
        pass

    def __str__(self):
        return f"{self.name()} with area {self.area():.2f}"

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def name(self):
        return "Circle"

    def area(self):
        return 3.14159 * self.radius ** 2

print(str(Circle(2))) # Circle with area 12.57
```

Magic Methods & Operator Overloading

058 Overload + and - for a Vector class

Create a 'Vector' class supporting addition and subtraction using `__add__` and `__sub__`.

SOLUTION – PYTHON

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __sub__(self, other):
        return Vector(self.x - other.x, self.y - other.y)

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"

v1 = Vector(2, 3)
v2 = Vector(1, 1)
print(v1 + v2)  # Vector(3, 4)
print(v1 - v2)  # Vector(1, 2)
```

059 Implement `__str__` and `__repr__` differently

Create a 'Product' class where `__str__` gives a friendly description and `__repr__` gives a developer-oriented, evaluable representation.

SOLUTION – PYTHON

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def __str__(self):
        return f"{self.name} costs ${self.price:.2f}"

    def __repr__(self):
        return f"Product(name={self.name!r}, price={self.price!r})"

p = Product("Mouse", 19.99)
print(str(p))    # Mouse costs $19.99
print(repr(p))   # Product(name='Mouse', price=19.99)
```

060 Implement full ordering with total_ordering

Create a 'Card' class ranked by value, implementing `__eq__` and `__lt__`, and use `functools.total_ordering` to get the rest of the comparisons for free.

SOLUTION – PYTHON

```
from functools import total_ordering

@total_ordering
class Card:
    def __init__(self, rank):
        self.rank = rank

    def __eq__(self, other):
        return self.rank == other.rank

    def __lt__(self, other):
        return self.rank < other.rank

    def __repr__(self):
        return f"Card({self.rank})"

cards = [Card(10), Card(2), Card(7)]
print(sorted(cards))          # [Card(2), Card(7), Card(10)]
print(Card(5) >= Card(3))     # True
```

061 Make an object indexable with `__getitem__` and `__setitem__`

Create a 'Matrix' class that supports indexing like `matrix[i][j]` using `__getitem__` and `__setitem__` on rows.

SOLUTION – PYTHON

```
class Matrix:
    def __init__(self, rows):
        self.rows = rows

    def __getitem__(self, index):
        return self.rows[index]

    def __setitem__(self, index, value):
        self.rows[index] = value

    def __repr__(self):
        return "\n".join(str(row) for row in self.rows)

m = Matrix([[1, 2], [3, 4]])
print(m[0])      # [1, 2]
m[1] = [9, 9]
print(m)
# [1, 2]
# [9, 9]
```

062 Custom iterable class with `__iter__` and `__next__`

Create a 'Fibonacci' class that generates Fibonacci numbers up to a limit using the iterator protocol.

SOLUTION – PYTHON

```
class Fibonacci:
    def __init__(self, limit):
        self.limit = limit

    def __iter__(self):
        self.a, self.b = 0, 1
        return self

    def __next__(self):
        if self.a > self.limit:
            raise StopIteration
        value = self.a
        self.a, self.b = self.b, self.a + self.b
        return value

for num in Fibonacci(20):
    print(num, end=" ")
# 0 1 1 2 3 5 8 13
```

063 Make an object callable with `__call__`

Create a 'Multiplier' class whose instances behave like functions that multiply their input by a stored factor.

SOLUTION – PYTHON

```
class Multiplier:
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, value):
        return value * self.factor

double = Multiplier(2)
triple = Multiplier(3)
print(double(5)) # 10
print(triple(5)) # 15
```

064 Context manager class using `__enter__` and `__exit__`

Create a 'Timer' context manager class that records and prints elapsed time when a 'with' block exits.

SOLUTION – PYTHON

```
import time

class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.elapsed = time.perf_counter() - self.start
        print(f"Elapsed: {self.elapsed:.4f} seconds")
        return False # don't suppress exceptions

with Timer():
    total = sum(range(1_000_000))
print(total)
```

065 Support 'in' operator with `__contains__`

Create a 'Playlist' class that supports 'song in playlist' membership checks using `__contains__`.

SOLUTION – PYTHON

```
class Playlist:
    def __init__(self, songs):
        self.songs = songs

    def __contains__(self, song):
        return song in self.songs

p = Playlist(["Song A", "Song B"])
print("Song A" in p)  # True
print("Song Z" in p)  # False
```

066 Overload `__eq__` and `__hash__` for use in sets

Create a 'Coordinate' class that can be safely stored in a set by implementing both `__eq__` and `__hash__` consistently.

SOLUTION – PYTHON

```
class Coordinate:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __eq__(self, other):
        return isinstance(other, Coordinate) and self.x ==
other.x and self.y == other.y

    def __hash__(self):
        return hash((self.x, self.y))

    def __repr__(self):
        return f"({self.x}, {self.y})"

points = {Coordinate(1, 2), Coordinate(1, 2), Coordinate(3,
4)}
print(points)  # {(1, 2), (3, 4)} (duplicate removed)
```

067 Overload `__bool__` for truthiness checks

Create a 'ShoppingCart' class that is considered 'falsy' when empty, by implementing `__bool__`.

SOLUTION – PYTHON

```
class ShoppingCart:
    def __init__(self):
        self.items = []

    def add(self, item):
        self.items.append(item)

    def __bool__(self):
        return len(self.items) > 0

cart = ShoppingCart()
print("Cart has items" if cart else "Cart is empty") # Cart is empty
cart.add("Book")
print("Cart has items" if cart else "Cart is empty") # Cart has items
```

068 Overload `__len__` and `__getitem__` together for a custom sequence

Create a 'Queue' class that behaves like a read-only sequence by supporting `len()` and indexing.

SOLUTION – PYTHON

```
class Queue:
    def __init__(self, items):
        self._items = list(items)

    def __len__(self):
        return len(self._items)

    def __getitem__(self, index):
        return self._items[index]

q = Queue([10, 20, 30])
print(len(q))    # 3
print(q[1])      # 20
print(list(q))   # [10, 20, 30]
```

069 Overload `__radd__` for reflected addition

Create a 'Money' class that supports `sum([Money(5), Money(10)])` by handling the case where `int (0)` is added on the left via `__radd__`.

SOLUTION – PYTHON

```
class Money:
    def __init__(self, amount):
        self.amount = amount

    def __add__(self, other):
        other_amount = other.amount if isinstance(other,
Money) else other
        return Money(self.amount + other_amount)

    def __radd__(self, other):
        return self.__add__(other)

    def __repr__(self):
        return f"${self.amount}"

total = sum([Money(5), Money(10), Money(3)])
print(total)  # $18
```

Static & Class Members

070 Static utility method

Create a 'MathUtils' class with a static method `is_prime()` that does not depend on any instance or class state.

SOLUTION – PYTHON

```
class MathUtils:
    @staticmethod
    def is_prime(n):
        if n < 2:
            return False
        for i in range(2, int(n ** 0.5) + 1):
            if n % i == 0:
                return False
        return True

print(MathUtils.is_prime(17)) # True
print(MathUtils.is_prime(18)) # False
```

071 Class method as an alternative constructor

Create a 'Date' class with a classmethod `from_string()` that parses a 'YYYY-MM-DD' string and returns a new Date instance.

SOLUTION – PYTHON

```
class Date:
    def __init__(self, year, month, day):
        self.year, self.month, self.day = year, month, day

    @classmethod
    def from_string(cls, date_str):
        year, month, day = map(int, date_str.split("-"))
        return cls(year, month, day)

    def __repr__(self):
        return f"{self.year:04d}-{self.month:02d}-{self.day:02d}"

d = Date.from_string("2026-08-04")
print(d)  # 2026-08-04
```

072 Class method returning the correct subclass

Create a 'Shape' base class with a classmethod default() that returns an instance of whatever subclass it's called on.

SOLUTION – PYTHON

```
class Shape:
    @classmethod
    def default(cls):
        return cls()

    def describe(self):
        return f"I am a {type(self).__name__}"

class Circle(Shape):
    pass

class Square(Shape):
    pass

print(Circle.default().describe()) # I am a Circle
print(Square.default().describe()) # I am a Square
```

073 Class attribute shared across instances

Create a 'Config' class with a shared class attribute 'settings' dictionary, and show that modifying it through one reference affects all instances.

SOLUTION – PYTHON

```
class Config:
    settings = {"debug": False}

c1 = Config()
c2 = Config()
Config.settings["debug"] = True
print(c1.settings["debug"]) # True
print(c2.settings["debug"]) # True (shared dict)
```

074 Instance counter using a class method

Extend a 'Widget' class to track total instances created, with a classmethod `get_count()` to retrieve the current count.

SOLUTION – PYTHON

```
class Widget:
    _count = 0

    def __init__(self):
        Widget._count += 1

    @classmethod
    def get_count(cls):
        return cls._count

Widget()
Widget()
Widget()
print(Widget.get_count()) # 3
```

075 Static method used for input validation before construction

Create a 'User' class with a static method `validate_username()` used inside `__init__` to check the username before assignment.

SOLUTION – PYTHON

```
class User:
    def __init__(self, username):
        if not User.validate_username(username):
            raise ValueError("Invalid username")
        self.username = username

    @staticmethod
    def validate_username(name):
        return name.isalnum() and len(name) >= 3

u = User("dev123")
print(u.username) # dev123
try:
    User("a!")
except ValueError as e:
    print(f"Error: {e}")
```

076 Class-level registry populated via classmethod

Create a 'Plugin' base class with a class-level list that every registered plugin subclass gets added to automatically via a classmethod.

SOLUTION – PYTHON

```
class Plugin:
    registry = []

    @classmethod
    def register(cls, plugin_cls):
        cls.registry.append(plugin_cls)
        return plugin_cls

@Plugin.register
class LoggerPlugin:
    pass

@Plugin.register
class CachePlugin:
    pass

print([p.__name__ for p in Plugin.registry])
# ['LoggerPlugin', 'CachePlugin']
```

077 Singleton implemented with a class attribute

Implement a simple Singleton class that reuses the same instance on every construction attempt by overriding `__new__`.

SOLUTION – PYTHON

```
class Singleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

a = Singleton()
b = Singleton()
print(a is b)  # True
```

Design Patterns

078 Singleton pattern with thread safety

Implement a thread-safe Singleton using a lock to guard instance creation.

SOLUTION – PYTHON

```
import threading

class Singleton:
    _instance = None
    _lock = threading.Lock()

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
                    cls._instance = super().__new__(cls)
        return cls._instance

a = Singleton()
b = Singleton()
print(a is b)  # True
```

079 Factory Method pattern

Implement a factory function that returns the correct Shape subclass instance based on a string type name.

SOLUTION – PYTHON

```
class Circle:
    def area(self):
        return "circle area"

class Square:
    def area(self):
        return "square area"

def shape_factory(shape_type):
    shapes = {"circle": Circle, "square": Square}
    if shape_type not in shapes:
        raise ValueError(f"Unknown shape type: {shape_type}")
    return shapes[shape_type]()

s = shape_factory("circle")
print(s.area()) # circle area
```

080 Builder pattern for constructing a complex object

Implement a Builder pattern for constructing a 'Pizza' object step by step with optional toppings.

SOLUTION – PYTHON

```
class Pizza:
    def __init__(self):
        self.toppings = []
        self.size = "medium"

    def __repr__(self):
        return f"{self.size} pizza with {', '.join(self.toppings) or 'no toppings'}"

class PizzaBuilder:
    def __init__(self):
        self.pizza = Pizza()

    def set_size(self, size):
        self.pizza.size = size
        return self

    def add_topping(self, topping):
        self.pizza.toppings.append(topping)
        return self

    def build(self):
        return self.pizza

pizza = (PizzaBuilder()
        .set_size("large")
        .add_topping("cheese")
        .add_topping("mushroom")
        .build())
print(pizza)  # large pizza with cheese, mushroom
```

081 Observer pattern

Implement the Observer pattern where a 'Publisher' notifies multiple 'Subscriber' objects when its state changes.

SOLUTION – PYTHON

```
class Publisher:
    def __init__(self):
        self._subscribers = []

    def subscribe(self, subscriber):
        self._subscribers.append(subscriber)

    def notify(self, message):
        for sub in self._subscribers:
            sub.update(message)

class Subscriber:
    def __init__(self, name):
        self.name = name

    def update(self, message):
        print(f"{self.name} received: {message}")

pub = Publisher()
pub.subscribe(Subscriber("Alice"))
pub.subscribe(Subscriber("Bob"))
pub.notify("New article published!")
# Alice received: New article published!
# Bob received: New article published!
```

082 Strategy pattern for interchangeable algorithms

Implement the Strategy pattern to compute a route using either 'FastestRoute' or 'ShortestRoute' strategies, chosen at runtime.

SOLUTION – PYTHON

```
class RouteStrategy:
    def calculate(self, start, end):
        raise NotImplementedError

class FastestRoute(RouteStrategy):
    def calculate(self, start, end):
        return f"Fastest route from {start} to {end} via highway"

class ShortestRoute(RouteStrategy):
    def calculate(self, start, end):
        return f"Shortest route from {start} to {end} via back roads"

class Navigator:
    def __init__(self, strategy: RouteStrategy):
        self.strategy = strategy

    def get_route(self, start, end):
        return self.strategy.calculate(start, end)

nav = Navigator(FastestRoute())
print(nav.get_route("A", "B"))
nav.strategy = ShortestRoute()
print(nav.get_route("A", "B"))
```

083 Decorator pattern for extending object behavior

Implement the Decorator pattern to add extra costs (milk, sugar) to a base 'Coffee' object without subclassing for every combination.

SOLUTION – PYTHON

```
class Coffee:
    def cost(self):
        return 3.0

    def description(self):
        return "Coffee"

class MilkDecorator:
    def __init__(self, coffee):
        self._coffee = coffee

    def cost(self):
        return self._coffee.cost() + 0.5

    def description(self):
        return self._coffee.description() + " + Milk"

class SugarDecorator:
    def __init__(self, coffee):
        self._coffee = coffee

    def cost(self):
        return self._coffee.cost() + 0.2

    def description(self):
        return self._coffee.description() + " + Sugar"

order = SugarDecorator(MilkDecorator(Coffee()))
print(order.description()) # Coffee + Milk + Sugar
print(order.cost())       # 3.7
```

084 Adapter pattern to bridge incompatible interfaces

Implement an Adapter that lets an old 'LegacyPrinter' class be used through a modern 'Printer' interface expected by client code.

SOLUTION – PYTHON

```
class LegacyPrinter:
    def old_print(self, text):
        return f"[LEGACY] {text}"

class Printer:
    def print(self, text):
        raise NotImplementedError

class LegacyPrinterAdapter(Printer):
    def __init__(self, legacy_printer):
        self.legacy_printer = legacy_printer

    def print(self, text):
        return self.legacy_printer.old_print(text)

def client_code(printer: Printer, text):
    print(printer.print(text))

client_code(LegacyPrinterAdapter(LegacyPrinter()), "Hello
World")
# [LEGACY] Hello World
```

085 Facade pattern simplifying a complex subsystem

Implement a 'HomeTheaterFacade' class that provides a single watchMovie() method hiding the complexity of coordinating a projector, sound system, and lights.

SOLUTION – PYTHON

```
class Projector:
    def on(self):
        return "Projector ON"

class SoundSystem:
    def on(self):
        return "Sound system ON"

class Lights:
    def dim(self):
        return "Lights dimmed"

class HomeTheaterFacade:
    def __init__(self):
        self.projector = Projector()
        self.sound = SoundSystem()
        self.lights = Lights()

    def watch_movie(self):
        steps = [self.lights.dim(), self.projector.on(),
self.sound.on()]
        return " -> ".join(steps)

theater = HomeTheaterFacade()
print(theater.watch_movie())
# Lights dimmed -> Projector ON -> Sound system ON
```

086 Command pattern with undo support

Implement the Command pattern for a simple text editor supporting an insert action and an undo() operation.

SOLUTION – PYTHON

```
class InsertTextCommand:
    def __init__(self, document, text):
        self.document = document
        self.text = text

    def execute(self):
        self.document.content += self.text

    def undo(self):
        self.document.content = self.document.content[: -
len(self.text)]

class Document:
    def __init__(self):
        self.content = ""

doc = Document()
cmd = InsertTextCommand(doc, "Hello")
cmd.execute()
print(doc.content)  # Hello
cmd.undo()
print(doc.content)  # (empty string)
```

087 Composite pattern for a file-system-like tree

Implement the Composite pattern with 'File' and 'Folder' classes so folders can contain files or other folders, and both support a common size() method.

SOLUTION – PYTHON

```
class File:
    def __init__(self, name, size):
        self.name = name
        self._size = size

    def size(self):
        return self._size

class Folder:
    def __init__(self, name):
        self.name = name
        self.children = []

    def add(self, item):
        self.children.append(item)

    def size(self):
        return sum(child.size() for child in self.children)

root = Folder("root")
root.add(File("a.txt", 100))
sub = Folder("subfolder")
sub.add(File("b.txt", 250))
root.add(sub)
print(root.size()) # 350
```

Composition & Mixins

088 Basic composition: Car has an Engine

Model a 'Car' class that is composed of an 'Engine' object rather than inheriting from it.

SOLUTION – PYTHON

```
class Engine:
    def __init__(self, horsepower):
        self.horsepower = horsepower

    def start(self):
        return f"Engine starting with {self.horsepower} HP"

class Car:
    def __init__(self, make, horsepower):
        self.make = make
        self.engine = Engine(horsepower) # composition

    def start(self):
        return f"{self.make}: {self.engine.start()}"

c = Car("Ford", 300)
print(c.start()) # Ford: Engine starting with 300 HP
```

089 Delegation through composition

Create a 'Playlist' class that delegates storage operations to an internal list, exposing only add() and play_all().

SOLUTION – PYTHON

```
class Playlist:
    def __init__(self):
        self._songs = []

    def add(self, song):
        self._songs.append(song)

    def play_all(self):
        for song in self._songs:
            print(f"Playing: {song}")

p = Playlist()
p.add("Song A")
p.add("Song B")
p.play_all()
# Playing: Song A
# Playing: Song B
```

090 Multiple mixins combined on one class

Create 'ComparableMixin' and 'PrintableMixin' classes that each add reusable behavior, then combine both into a single 'Product' class.

SOLUTION – PYTHON

```
class ComparableMixin:
    def __eq__(self, other):
        return self.value == other.value

    def __lt__(self, other):
        return self.value < other.value

class PrintableMixin:
    def print_info(self):
        print(f"{type(self).__name__}: {self.value}")

class Product(ComparableMixin, PrintableMixin):
    def __init__(self, value):
        self.value = value

p1 = Product(10)
p2 = Product(20)
p1.print_info()           # Product: 10
print(p1 < p2)            # True
```

091 Composition-based validation pipeline

Build a 'Form' class composed of independent Validator objects (e.g., NotEmptyValidator, MaxLengthValidator) that are each applied to the input.

SOLUTION – PYTHON

```
class NotEmptyValidator:
    def validate(self, value):
        return len(value) > 0

class MaxLengthValidator:
    def __init__(self, max_len):
        self.max_len = max_len

    def validate(self, value):
        return len(value) <= self.max_len

class Form:
    def __init__(self, validators):
        self.validators = validators

    def is_valid(self, value):
        return all(v.validate(value) for v in
self.validators)

form = Form([NotEmptyValidator(), MaxLengthValidator(10)])
print(form.is_valid("hello"))      # True
print(form.is_valid(""))          # False
print(form.is_valid("way too long")) # False
```

092 Strategy injected via composition instead of inheritance

Instead of subclassing a 'Sorter' for each algorithm, inject a sorting function into the Sorter's constructor.

SOLUTION – PYTHON

```
class Sorter:
    def __init__(self, strategy):
        self.strategy = strategy

    def sort(self, data):
        return self.strategy(data)

ascending = Sorter(sorted)
descending = Sorter(lambda data: sorted(data, reverse=True))

print(ascending.sort([3, 1, 2])) # [1, 2, 3]
print(descending.sort([3, 1, 2])) # [3, 2, 1]
```

093 Mixin adding serialization to multiple unrelated classes

Create a 'DictSerializableMixin' that any class can inherit from to gain a to_dict() method based on its __dict__.

SOLUTION – PYTHON

```
class DictSerializableMixin:
    def to_dict(self):
        return dict(self.__dict__)

class User(DictSerializableMixin):
    def __init__(self, name, email):
        self.name = name
        self.email = email

class Order(DictSerializableMixin):
    def __init__(self, order_id, total):
        self.order_id = order_id
        self.total = total

print(User("Ana", "ana@example.com").to_dict())
print(Order(101, 59.99).to_dict())
```

094 Composing a Logger into unrelated services

Instead of every service class inheriting from a `LoggerMixin`, inject a shared 'Logger' object into each service via composition.

SOLUTION – PYTHON

```
class Logger:
    def log(self, message):
        print(f"[LOG] {message}")

class OrderService:
    def __init__(self, logger):
        self.logger = logger

    def place_order(self, order_id):
        self.logger.log(f"Order {order_id} placed")

class PaymentService:
    def __init__(self, logger):
        self.logger = logger

    def process_payment(self, amount):
        self.logger.log(f"Payment of ${amount} processed")

logger = Logger()
OrderService(logger).place_order(101)
PaymentService(logger).process_payment(49.99)
```

095 Has-a relationship modeled with a list of composed objects

Model a 'House' class composed of multiple 'Room' objects, with a method to compute the total area of the house.

SOLUTION – PYTHON

```
class Room:
    def __init__(self, name, area):
        self.name = name
        self.area = area

class House:
    def __init__(self):
        self.rooms = []

    def add_room(self, room):
        self.rooms.append(room)

    def total_area(self):
        return sum(room.area for room in self.rooms)

house = House()
house.add_room(Room("Bedroom", 20))
house.add_room(Room("Kitchen", 15))
print(house.total_area()) # 35
```

CHAPTER 10

Exception Handling in OOP

096 Custom exception class

Create a custom 'InsufficientFundsError' exception and raise it from a BankAccount's withdraw() method.

SOLUTION – PYTHON

```
class InsufficientFundsError(Exception):
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
        super().__init__(f"Cannot withdraw {amount}: balance
is only {balance}")

class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientFundsError(self.balance,
amount)
        self.balance -= amount

acc = BankAccount(50)
try:
    acc.withdraw(100)
except InsufficientFundsError as e:
    print(f"Error: {e}")
# Error: Cannot withdraw 100: balance is only 50
```

097 Exception hierarchy for a custom library

Design a small exception hierarchy: a base 'ValidationError' with two subclasses 'RequiredFieldError' and 'InvalidFormatError'.

SOLUTION – PYTHON

```
class ValidationError(Exception):
    pass

class RequiredFieldError(ValidationError):
    def __init__(self, field):
        super().__init__(f"&#x27;{field}' is required")

class InvalidFormatError(ValidationError):
    def __init__(self, field, value):
        super().__init__(f"&#x27;{field}' has invalid value: {value}")

def validate(data):
    if "email" not in data:
        raise RequiredFieldError("email")
    if "@" not in data["email"]:
        raise InvalidFormatError("email", data["email"])

try:
    validate({"email": "bademail"})
except ValidationError as e:
    print(f"Validation failed: {e}")
```

098 Exception chaining with 'raise ... from'

Catch a low-level exception and re-raise a more descriptive custom exception while preserving the original cause.

SOLUTION – PYTHON

```
class DataProcessingError(Exception):
    pass

def parse_number(value):
    try:
        return int(value)
    except ValueError as original_error:
        raise DataProcessingError(f"Could not parse {value} as a number") from original_error

try:
    parse_number("abc")
except DataProcessingError as e:
    print(f"Error: {e}")
    print(f"Caused by: {e.__cause__}")
# Error: Could not parse 'abc' as a number
# Caused by: invalid literal for int() with base 10: 'abc'
```

099 Context manager guaranteeing cleanup even on exception

Create a 'ManagedResource' context manager class that always releases a resource in `__exit__`, even if the code inside the 'with' block raises an exception.

SOLUTION – PYTHON

```
class ManagedResource:
    def __enter__(self):
        print("Resource acquired")
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Resource released")
        return False # propagate any exception

try:
    with ManagedResource():
        raise RuntimeError("Something went wrong")
except RuntimeError as e:
    print(f"Caught: {e}")
# Resource acquired
# Resource released
# Caught: Something went wrong
```

100 Catching a base exception type to handle multiple subclasses generically

Define two specific exceptions that both inherit from a shared 'AppError' base, and catch them generically in one except block.

SOLUTION – PYTHON

```
class AppError(Exception):
    pass

class NetworkError(AppError):
    pass

class TimeoutError_(AppError):
    pass

def risky_operation(kind):
    if kind == "network":
        raise NetworkError("Connection refused")
    elif kind == "timeout":
        raise TimeoutError_("Request timed out")

for kind in ("network", "timeout"):
    try:
        risky_operation(kind)
    except AppError as e:
        print(f"Handled {type(e).__name__}: {e}")
# Handled NetworkError: Connection refused
# Handled TimeoutError_: Request timed out
```

– End of Handbook –